

GLIMS (UPV) - Implementation Roadmap

Greening City Logistics · Innovative Sustainable Last-Mile Solutions for Barcelona

How to read this document

The roadmap is organised in **three sequential phases**. Each phase has a single objective and a set of tasks. Tasks are ordered by dependency, so working top to bottom keeps the build coherent. Each task lists **what** it changes, **where** it lives (file and function names, not full code), and the **issue numbers** from our earlier findings list so nothing is lost.

Legend: ■ code / model change ■ experiment / validation ■ writing / documentation

The three phases

Phase	Objective	Outcome
Phase 1 · Solid Foundations	Fix critical model bugs, unify the codebase, make it reproducible.	One trustworthy, portable simulator.
Phase 2 · Algorithmic & Experimental	Align the method with the project's LRP + AI framing and add rigor.	Optimisation core + designed experiments.
Phase 3 · Prototype to Paper	Complete experiments and write the manuscript.	Submission-ready article.

Phase 1 · Solid Foundations. This phase brings order to what already exists. The scattered logic is unified into a single module, the three modeling bugs that currently distort the results are fixed, all parameters are centralized into one source of truth, and the code is made portable and reproducible. The goal isn't to add anything new, but to have one reliable simulator that can be trusted before building anything on top of it.

Phase 2 · Algorithmic & Experimental Enhancements. Here the project moves from a descriptive scenario calculator to the optimization-based methodology the spec promises. The current approximations (rectangular boxes, hub at the centroid, nearest-neighbor routing) are replaced with real polygons, optimal hub location, and an optimization core built on Clarke-Wright + ILS. It's also surrounded with the rigor a reviewer expects: demand variability with confidence intervals, a sensitivity analysis, distance validation, and the full set of indicators. This is what turns the work into a research contribution.

Phase 3 · From Prototype to Paper. With a trustworthy method and solid results in hand, this phase is about analysis and writing. The results are cross-analyzed by urban density, reproducibility is locked down, and the manuscript is written section by section — from the PRISMA related-work review and the formal methodology through to the discussion, limitations, and policy conclusions. The code is essentially frozen; the focus shifts to turning reliable results into a defensible narrative ready for submission.

Phase 1 · Solid Foundations

Goal: fix the critical model bugs, collapse the three duplicated copies into one module, and make every run portable and reproducible. Nothing else can be trusted until this is done.

Everything downstream depends on a single, correct simulator.

1.1 Unify logic into one parameterised module

CODE *issues 5, 17*

What Collapse the three copies of the model (Colab notebook, the missing `simulador.py`, and `simulador_osmnx.py`) into a single reusable module. Notebooks only orchestrate and plot.

Where `simulator.py` as the home for `simulate_city` / `simulate_neighborhood` / `load_data`. Replaces scattered logic across all three files.

1.2 Fix the three model inconsistencies

CODE *issues 1, 2, 3*

What Internal route must scale with the number of trips (split demand into capacity-sized batches); the resupply van in green scenarios must use the electric vehicle, not diesel; customer car travel must not reuse the pedestrian distance.

Where `simulate_neighborhood` — the M1/M2 trip logic, the shared `resupply van` cost/CO₂ terms, and the M5 customer-CO₂ term.

1.3 One source of truth for parameters

CODE *issues 4, 18*

What All model constants (cost, speed, emissions, capacity, fees) live in one CSV and are read from it everywhere. Remove hardcoded copies from notebooks. Reconcile capacity and hub size with the project spec.

Where `model_parameters.csv` consumed by `load_parameters`. Removes literals from the notebook and from `PARAMETROS_MODELLOS`.

1.4 Portable paths

CODE *issues 6*

What Replace the absolute Windows paths with project-relative paths so the code runs on any machine.

Where Path constants at the top of `preparar_datos.py` — derive from `BASE_DIR` or a small config.

1.5 Clean code & logging

CODE *issues 9*

What Remove dead/commented blocks and debug prints; replace prints with configurable logging so runs stay quiet by default.

Where Throughout `simulador_osmnx.py` — the commented distance helper, leftover seed line, and inline debug prints.

1.6 Pin the environment

CODE *issues 10*

What Freeze dependency versions (OSMnx above all, whose function signatures changed between major versions) so the code doesn't silently break.

Where `requirements.txt` / `environment.yml` at the project root.

1.7 Source every assumption

WRITING *issues 12, 13, 14*

What Attach a reference or a documented rationale to each emission factor, operating cost, and geometric heuristic (the bike-detour factor and the radial approximation). This is what turns 'magic numbers' into defensible inputs.

Where Alongside `model_parameters.csv` as an annotated parameter sheet; feeds the paper's data section.

Exit criteria: one module, no duplicated logic, no absolute paths, a pinned environment, and a parameter sheet where every number has a source.

Phase 2 · Algorithmic & Experimental Enhancements

Goal: move from a descriptive scenario calculator to the optimisation-based, AI-framed method the project promises — and surround it with the rigor a reviewer expects.

This phase is what makes the work a research contribution, not a spreadsheet.

2.1 Real neighborhood polygons

CODE issues 7

What Replace the rectangular bounding boxes (which overlap and leave gaps) with real administrative polygons and point-in-polygon filtering.

Where `filter_neighborhood_points` reading polygons obtained the way `consultas.ipynb` already explores them.

2.2 Efficient distances

CODE issues 8

What Compute only the distances actually needed (a single centroid row for the radial models; a compact client-to-client matrix for routing) and cache them, instead of a full dense matrix per neighborhood.

Where `build_distance_matrix` / `get_neighborhood_matrices` in `simulator.py`.

2.3 Locate the hub/PUDO (the 'L' in LRP)

CODE issues 19

What Stop assuming the hub sits at the centroid. Choose its location among real candidate sites (car parks, markets, post offices, metro, lockers) — this is the core of research questions PR1–PR2.

Where New `locate_hub` step feeding `simulate_neighborhood`; replaces the centroid assumption in `prepare_neighborhood`.

2.4 Optimisation routing: Clarke-Wright + ILS

CODE issues 16, 20

What Replace nearest-neighbor with the method the spec defines: Clarke-Wright savings (multi-start) for initial solutions, improved by an Iterated Local Search metaheuristic, under an objective that also penalises emissions and vulnerable zones.

Where New `savings_multistart` and `iterated_local_search` replacing `tsp_nearest_neighbor`.

2.5 Demand variability & repetitions

CODE issues 21

What Turn the single deterministic run into many: sample demand/volume scenarios, repeat, and aggregate with means and confidence intervals. This is the bridge to the project's Monte Carlo framing.

Where Scenario loop wrapping `simulate_city`; fixed seeds for reproducibility.

2.6 Sensitivity analysis

EXPERIMENT issues 22

What Sweep the influential parameters (fuel cost, emission factor, capacity, PUDO fee, hub cost) and report how the winning model changes. Shows the conclusions aren't an artefact of one guess.

Where New analysis routine `sensitivity_analysis` in the analysis notebook/module.

2.7 Validate the distances

EXPERIMENT issues 23, 15

What Contrast a sample of OSM routes against an independent source (OSRM/Google) and report the error, so the routing backbone is trusted.

Where New `validate_distances` routine; results feed the methodology/limitations.

2.8 Complete the metrics

CODE issues 24

What Report all ten indicators the spec asks for: kilometres and trips split by mode (van, bike, delivery on foot, customer on foot), plus emissions and cost, per neighborhood and city.

Where Output schema of each model block inside `simulate_neighborhood`.

Exit criteria: an optimisation core (location + CWS/ILS routing), stochastic runs with intervals, validated distances, a sensitivity study, and the full indicator set.

Phase 3 · From Prototype to Paper

Goal: run the final experiments and write the manuscript. Most of this is analysis and writing; the code is essentially frozen.

Turning trustworthy results into a defensible narrative.

3.1 Urban density analysis (PR3)

EXPERIMENT *issues 25*

What Cross the results with each neighborhood's demand/population density to explain how cost and emissions shift from dense (Eixample) to sparse (El Pardo) areas.

Where New `analyze_density` step in the analysis notebook.

3.2 Lock reproducibility

EXPERIMENT *issues 26*

What Fix seeds, keep the pinned versions, and save the exact configuration next to every results file so any run can be reproduced.

Where Result-writing path in the analysis notebook; config dumped beside each CSV.

3.3 Systematic related-work review

WRITING *issues 27*

What Run the PRISMA review the spec already promises (WoS/Scopus) on microhubs, PUDO, cargo-bikes, and last-mile LRP, and write the related-work section.

Where Manuscript — Related Work.

3.4 Formal methodology section

WRITING *issues 28*

What Write the mathematical formulation: the LRP, the penalised objective function, and the CWS + ILS algorithms — mirroring the code from Phase 2.

Where Manuscript — Methodology.

3.5 Formal data description

WRITING *issues 29*

What Describe the dataset: origin, volume per neighborhood/city, cleaning steps, and representativeness.

Where Manuscript — Data; draws on `clean_dataframe` in `preparar_datos.py`.

3.6 Experimental design & consolidated results

WRITING *issues 30, 31*

What State the experimental protocol (factors, repetitions, metrics, significance) and consolidate the scattered exploratory plots into final figures and tables.

Where Manuscript — Experiments & Results; from the analysis notebook.

3.7 Discussion, limitations, conclusions

WRITING *issues 32, 33, 34*

What Interpret which model wins where and why and the cost–emissions trade-off; acknowledge the simplifications honestly; close with conclusions and PMUS/E-DUM policy implications.

Where Manuscript — Discussion, Limitations, Conclusions.

Exit criteria: a complete manuscript with reproducible experiments, consolidated figures, and an honest discussion — ready for submission (Transportation Research Part A target).

Sequencing at a glance

Read left to right. Each phase gates the next; within a phase, tasks flow top to bottom by dependency.

PHASE 1 unify → fix bugs → central params → paths → clean → pin env → source assumptions

|

PHASE 2 polygons → fast distances → locate hub → CWS/ILS → variability → sensitivity → validate → full metrics

|

PHASE 3 density → reproducibility → PRISMA → methodology → data → experiments/results → discussion

Immediate next step: Phase 1, Task 1.1 — draft the structure of the unified `simulator.py` module. Once agreed, each task becomes a tracked issue.